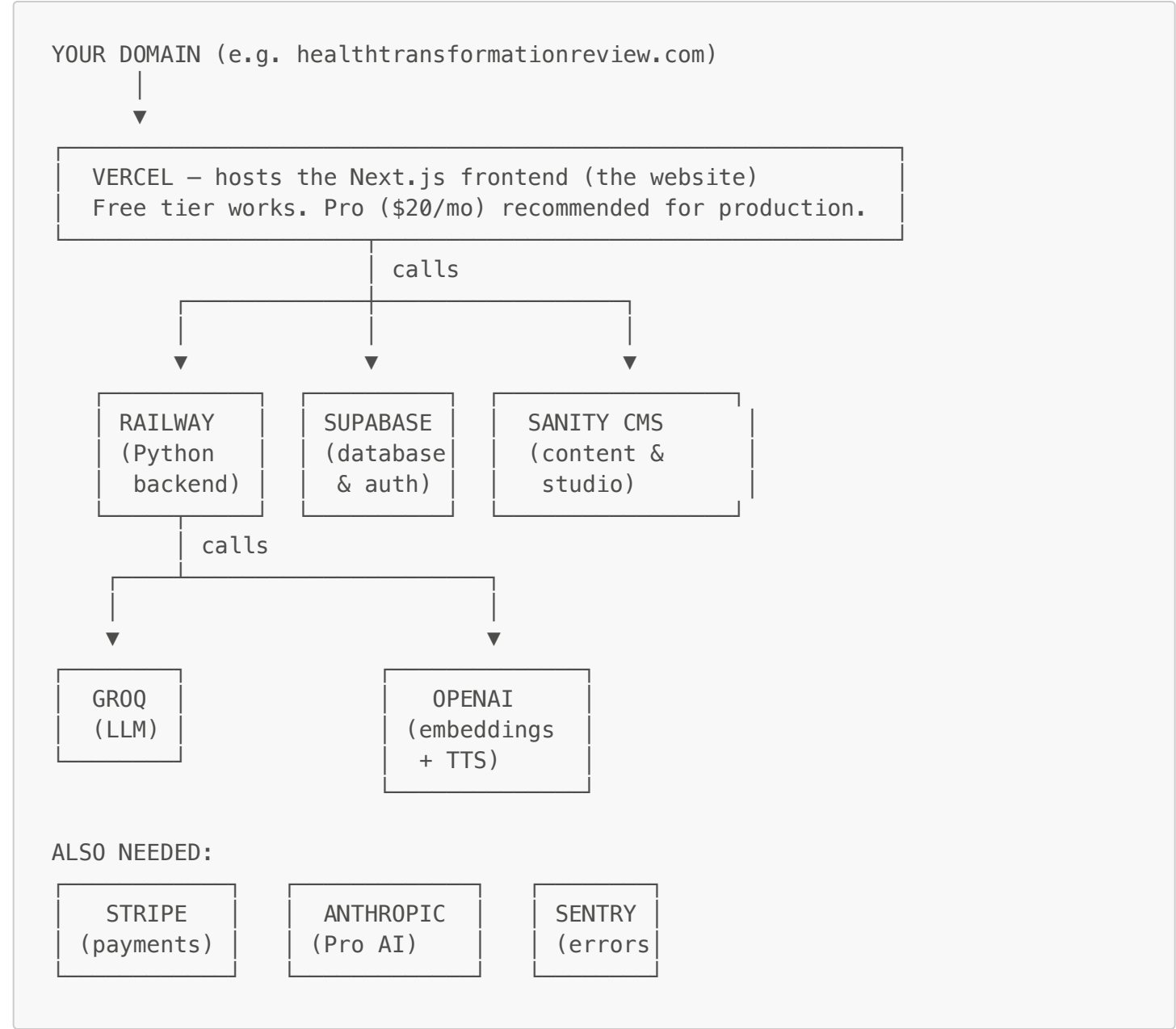


Deployment Guide — Vermont Health Platform (HTR)

Audience: Anyone deploying this platform for the first time. No prior DevOps experience assumed. **Version:** 4.2.0
Time to complete: 2–4 hours for a full first deployment.

Overview — What You Are Deploying

This platform has **eight external services** that must all be set up and connected before it works. You will create accounts on each, collect credentials (API keys, secrets, URLs), and wire them together through environment variables.



Deployment order matters. Set up services in the order in this guide. Each section tells you exactly what to collect before moving on.

Table of Contents

1. [Prerequisites](#)
2. [GitHub — Repository Setup](#)
3. [Supabase — Database & Auth](#)
4. [Sanity — Content Management](#)
5. [Groq — LLM Inference](#)
6. [OpenAI — Embeddings & TTS](#)
7. [Anthropic — Pro Tier AI](#)
8. [Stripe — Payments](#)
9. [Sentry — Error Monitoring](#)
10. [Railway — Python Backend](#)

- 11. [Vercel — Frontend](#)
- 12. [Connect Everything — Environment Variables](#)
- 13. [Post-Deployment — Webhooks & Final Setup](#)
- 14. [Verify Everything Works](#)
- 15. [Custom Domain Setup](#)
- 16. [Troubleshooting](#)

1. Prerequisites

Accounts You Need to Create

Create accounts at each of these before starting. All have free tiers that work for initial setup.

Service	URL	Free Tier	Paid Tier Needed?
GitHub	github.com	Yes	No
Supabase	supabase.com	Yes (2 projects)	Pro (\$25/mo) for production backups
Sanity	sanity.io	Yes	No
Groq	console.groq.com	Yes (rate-limited)	Pay-as-you-go for production
OpenAI	platform.openai.com	No (credit required)	Pay-as-you-go
Anthropic	console.anthropic.com	No (credit required)	Pay-as-you-go
Stripe	stripe.com	Yes (test mode)	Required for payments
Sentry	sentry.io	Yes (5k errors/mo)	No for initial deployment
Railway	railway.app	Yes (\$5 credit)	Hobby (\$5/mo) minimum
Vercel	vercel.com	Yes	Pro (\$20/mo) recommended

Tools to Install on Your Computer

```
# Node.js (version 20 or higher)
# Download from: nodejs.org

# Verify installation:
node --version      # should show v20.x.x or higher
npm --version       # should show 9.x or higher

# Git
git --version       # should show any version

# Python (version 3.10 or higher)
python3 --version   # should show 3.10.x or higher
```

If any of these are missing, install them before continuing.

2. GitHub — Repository Setup

You need the code in a GitHub repository that Vercel and Railway can connect to.

If you have the code locally

```
# Navigate to the project folder
cd Vermont-Health-Platform

# Initialize git if not already done
```

```
git init
git add .
git commit -m "Initial commit"

# Create a new repository on github.com
# Then push:
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPO-NAME.git
git branch -M main
git push -u origin main
```

If the code is already on GitHub

You are ready. Note the repository URL — you will need it when connecting to Vercel and Railway.

3. Supabase — Database & Auth

Supabase provides the PostgreSQL database, authentication system, and vector store. This is the most involved setup step.

Step 1 — Create a Project

1. Go to [supabase.com](#) → Sign in → New Project
2. Choose an organization (or create one)
3. **Project name:** `htr-production` (or any name)
4. **Database password:** Generate a strong password and **save it somewhere safe** — you will need it
5. **Region:** Choose the region closest to your users (US East for Vermont-focused platform)
6. Click **Create new project** — takes 1–2 minutes to provision

Step 2 — Enable the pgvector Extension

The AI system stores vector embeddings in the database. pgvector must be enabled first.

1. In your Supabase dashboard → **Database** (left sidebar) → **Extensions**
2. Search for `vector`
3. Click the toggle to enable it
4. Confirm when prompted

Step 3 — Run the Database Schema

The platform requires specific tables, indexes, and functions. Run all migration files in order.

1. In Supabase dashboard → **SQL Editor** (left sidebar)
2. Click **New query**
3. Open each file in `supabase/migrations/` on your computer, copy the contents, paste into the SQL editor, click **Run**
4. Run them in numerical order:

```
001_profiles_and_roles.sql      ← Run first
002_content_data.sql
003_academy.sql
004_advisory.sql
005_rag_vectors.sql             ← This enables pgvector tables
006_rls_policies.sql
007_hybrid_search.sql           ← This creates the AI search function
008_rls_audit.sql
009_referrals.sql
010_community.sql
011_api_keys.sql
012_survey.sql
013_role_audit.sql
014_webhook_inbox.sql
```

```
015_rag_query_log.sql
016_bookmarks.sql
017_ticker_cache.sql
018_role_change_log.sql
019_user_learning_paths.sql    ← Run last
```

After each file runs, you should see **"Success. No rows returned"** or similar. If you see an error, check that pgvector was enabled in Step 2 and that you are running files in order.

Step 4 — Collect Your Credentials

Go to **Project Settings** (gear icon, bottom left) → **API**

Write down or copy these — you need all of them:

Project URL:

https://[your-project-id].supabase.co
(looks like: https://abcdefghijklm.supabase.co)

anon/public key:

eyJ... (long string starting with eyJ)
This is safe to expose in the browser.

service_role key:

eyJ... (different long string starting with eyJ)
KEEP THIS SECRET. Never put in browser-visible code.

JWT Secret:

(under "JWT Settings" section)
A long random string. KEEP THIS SECRET.

Go to **Project Settings** → **Database** → **Connection string** → select **URI**:

Direct connection URL:

postgresql://postgres:[YOUR-DB-PASSWORD]@db.[project-id].supabase.co:5432/postgres

Save this – the backend needs it for pgvector operations.

Step 5 — Configure Auth Settings

1. In Supabase dashboard → **Authentication** → **URL Configuration**
2. Set **Site URL** to your production domain (e.g., <https://healthtransformationreview.com>)
 - If you don't have a domain yet, use your Vercel URL after deploying (come back to this)
3. Under **Redirect URLs**, add:
 - https://your-domain.com/**
 - https://your-vercel-url.vercel.app/** (for preview deploys)
4. Click **Save**

What You Have Now

NEXT_PUBLIC_SUPABASE_URL

= https://[project-id].supabase.co

NEXT_PUBLIC_SUPABASE_ANON_KEY

= eyJ... (anon key)

SUPABASE_SERVICE_ROLE_KEY

= eyJ... (service role key)

SUPABASE_URL

= https://[project-id].supabase.co (same as
above, for backend)

SUPABASE_JWT_SECRET

= (JWT secret from API settings)

SUPABASE_DB_URL

= postgresql://postgres:...@db.[project-id].supabase.co:5432/postgres

4. Sanity — Content Management

Sanity is the headless CMS where all articles, courses, and content live. The platform uses an existing Sanity project ID (`fxz10xl7`). If you are starting fresh with your own Sanity project, follow these steps.

Step 1 — Create or Access Your Sanity Project

Option A — Use the existing project (if you have access to `fxz10xl7`):

1. Go to sanity.io/manage
2. Find the project `fxz10xl7`
3. Go to **API** → **Tokens** → **Add API token**
4. Name it `HTR Production`, set permissions to **Editor**, click **Save**
5. Copy the token — it starts with `sk` and you only see it once

Option B — Create a new Sanity project:

1. Go to sanity.io/manage → **Create new project**
2. Choose a name, select the **Free** plan
3. Note the **Project ID** shown on the project page
4. Go to **API** → **Datasets** → confirm `production` dataset exists (create it if not)
5. Go to **API** → **Tokens** → **Add API token** → name it `HTR Production` → **Editor** permissions → **Save**
6. Copy the token

Step 2 — Update the Project ID in Code

If you created a new project (Option B), update this file:

```
// frontend/lib/sanity/client.ts (or wherever the projectId is configured)
// Change fxz10xl7 to your new project ID
```

Also update `backend/.env`:

```
SANITY_PROJECT_ID=your-new-project-id
```

Step 3 — Generate a Webhook Secret

This is a random string you create yourself — it is used to verify that webhook calls actually come from Sanity.

```
# Generate a random secret (run this in your terminal):
openssl rand -base64 32
```

Copy the output. You will use it in two places: Sanity webhook config and your Vercel environment variables.

What You Have Now

```
NEXT_PUBLIC_SANITY_PROJECT_ID = fxz10xl7 (or your new project ID)
NEXT_PUBLIC_SANITY_DATASET    = production
SANITY_API_TOKEN              = sk... (the token you created)
SANITY_WEBHOOK_SECRET         = (the random string you generated)
```

For the backend:

```
SANITY_PROJECT_ID    = fxz10xl7
SANITY_DATASET        = production
```

```
SANITY_API_TOKEN      = sk...  (same token)
SANITY_API_VERSION    = 2023-10-01
```

5. Groq — LLM Inference

Groq provides fast LLM inference for the AI Analyst (subscriber and free tier users use Llama 3.3-70b via Groq).

Step 1 — Create an Account and Get an API Key

1. Go to [console.groq.com](#) → Sign up
 2. Once logged in → **API Keys** (left sidebar) → **Create API Key**
 3. Name it **HTR Production**
 4. Copy the key — it starts with **gsk_**

Step 2 — Set a Usage Limit (Recommended)

1. In Groq console → **Settings** → **Usage Limits**
 2. Set a monthly dollar limit to prevent unexpected costs
 3. Start with \$20–\$50/month for a new deployment; adjust based on usage

What You Have Now

```
GROQ_API_KEY = gsk_...
```

6. OpenAI — Embeddings & TTS

OpenAI is used for two things: creating vector embeddings when content is indexed (required for AI search to work) and text-to-speech audio for the personalized learning feature.

Step 1 — Create an Account and Add Credits

1. Go to [platform.openai.com](#) → Sign up
 2. Go to **Billing** → **Add payment method** → add a credit card
 3. Go to **Billing** → **Buy credits** → add at least \$10 to start
 - Embedding costs are low (~\$0.02 per million tokens)
 - TTS costs more (~\$15 per million characters)

Step 2 — Create an API Key

1. Go to **API keys** (left sidebar) → **Create new secret key**
 2. Name it **HTR Production**
 3. Copy the key — it starts with **sk-**
 4. **You cannot see this key again** — save it immediately

Step 3 — Set Usage Limits

1. Go to **Settings** → **Limits**
 2. Set a monthly usage limit (start with \$20)

What You Have Now

```
OPENAI_API_KEY = sk-...
```

7. Anthropic — Pro Tier AI

Anthropic's Claude is used only for Professional and Advisory tier users, who get higher-quality responses. This is optional for initial launch but required if you want the Pro tier to work.

Step 1 — Create an Account

1. Go to console.anthropic.com → Sign up
2. Go to **Billing** → add a payment method and buy credits (start with \$20)

Step 2 — Create an API Key

1. Go to **API Keys** → **Create Key**
2. Name it **HTR Production**
3. Copy the key — starts with **sk-ant-**

What You Have Now

```
ANTHROPIC_API_KEY = sk-ant-...
```

8. Stripe — Payments

Stripe handles subscription billing. You work in **test mode** first, then switch to **live mode** for real payments.

Step 1 — Create an Account

1. Go to stripe.com → **Start now** → Sign up
2. Complete your business profile (required even for test mode)
3. You will see a dashboard — make sure the toggle in the top right says **Test mode**

Step 2 — Create Products and Prices

You need to create the subscription products that users will purchase.

In Stripe dashboard → **Product catalog** → **Add product**:

Product 1: Subscriber

- Name: **HTR Subscriber**
- Pricing model: **Recurring**
- Price: **\$29.00** USD / month → click **Add price** → note the Price ID (starts with **price_**)
- Add another price: **\$290.00** USD / year → note that Price ID too

Product 2: Student

- Name: **HTR Student**
- Monthly: **\$19.00** / month → note Price ID
- Annual: **\$190.00** / year → note Price ID

Product 3: Professional

- Name: **HTR Professional**
- Monthly: **\$99.00** / month → note Price ID
- Annual: **\$990.00** / year → note Price ID

Step 3 — Get Your API Keys

In Stripe dashboard → **Developers** → **API keys**:

```
Publishable key: pk_test_... (safe for browser)
Secret key:      sk_test_... (keep secret – server only)
```

You will switch these to `pk_live_` and `sk_live_` when going live.

Step 4 — Create a Webhook (Do This After Vercel is Deployed)

You cannot do this step until you have your production URL. Skip for now — return here in Step 13.

What You Have Now

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY = pk_test...
STRIPE_SECRET_KEY                  = sk_test...
STRIPE_PRICE_SUBSCRIBER_MONTHLY    = price...
STRIPE_PRICE_SUBSCRIBER_ANNUAL     = price...
STRIPE_PRICE_PROFESSIONAL_MONTHLY  = price...
STRIPE_PRICE_PROFESSIONAL_ANNUAL   = price...
```

9. Sentry — Error Monitoring

Sentry captures errors from both the frontend and backend so you know when things break in production.

Step 1 — Create an Account and Project

1. Go to [sentry.io](#) → Sign up
2. Create a new **Organization**
3. Create a new **Project**:
 - Platform: **Next.js**
 - Name: `htr-frontend`
 - Click **Create Project**
4. Sentry will show you a DSN — copy it (looks like `https://abc123@o123.ingest.sentry.io/456`)
5. Create a **second project**:
 - Platform: **Python**
 - Name: `htr-backend`
 - Copy that DSN too

Step 2 — Create an Auth Token

1. In Sentry → **Settings** → **Auth Tokens** → **Create New Token**
2. Permissions needed: `project:write,org:read`
3. Copy the token

What You Have Now

```
NEXT_PUBLIC_SENTRY_DSN = https://...@sentry.io/... (frontend project DSN)
SENTRY_DSN             = https://...@sentry.io/... (backend project DSN –
different!)
SENTRY_AUTH_TOKEN      = ...
SENTRY_ORG             = your-org-slug
SENTRY_PROJECT         = htr-frontend
```

10. Railway — Python Backend

Railway hosts the FastAPI Python backend that powers the AI Analyst, RAG pipeline, and text-to-speech.

Step 1 — Create an Account

1. Go to [railway.app](#) → Sign up with GitHub (recommended — makes repo connection easier)
2. You get \$5 in free credits. Upgrade to **Hobby** (\$5/month) for a production deployment — the free trial credits run out.

Step 2 — Create a New Project

1. In Railway dashboard → **New Project**
2. Select **Deploy from GitHub repo**
3. Connect your GitHub account if not already connected
4. Select your repository
5. Railway will detect the repo — **do not deploy yet**

Step 3 — Configure the Service

1. Click on the service Railway created
2. Go to **Settings** tab
3. Under **Source**:
 - **Root Directory:** `backend`
 - **Build Command:** leave empty (Railway detects Python automatically)
 - **Start Command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`
4. Under **Networking** → **Generate Domain** → Railway gives you a URL like `htr-backend.up.railway.app`
 - **Copy this URL** — you need it for Vercel's `PYTHON_BACKEND_URL`

Step 4 — Generate the Ingest Secret

This is a shared secret between the frontend and backend — anyone with this secret can trigger an AI re-index. Generate it yourself:

```
openssl rand -base64 32
```

Copy the output. You will set it as `INGEST_SECRET` in **both** Vercel and Railway.

Step 5 — Add Environment Variables to Railway

In your Railway service → **Variables** tab → add each variable:

```
GROQ_API_KEY          = gsk_...
OPENAI_API_KEY         = sk-...
ANTHROPIC_API_KEY      = sk-ant-...
SUPABASE_URL           = https://[project-id].supabase.co
SUPABASE_SERVICE_ROLE_KEY = eyJ...
SUPABASE_JWT_SECRET    = (from Supabase Project Settings → API → JWT Settings)
SUPABASE_DB_URL        = postgresql://postgres:...@db.[project-id].supabase.co:5432/postgres
SANITY_PROJECT_ID      = fxz10xl7
SANITY_DATASET         = production
SANITY_API_TOKEN       = sk...
SANITY_API_VERSION     = 2023-10-01
INGEST_SECRET          = (the random string you generated)
FRONTEND_URL           = https://your-vercel-domain.vercel.app (update after Vercel deploy)
SENTRY_DSN             = https://...@sentry.io/... (backend Sentry DSN)
ENVIRONMENT            = production
```

Do NOT set PORT — Railway sets this automatically.

Step 6 — Deploy

1. In Railway → **Deploy** tab → **Deploy Now**
2. Watch the logs — first deploy takes 3–5 minutes to install Python dependencies
3. First startup after that takes 5–15 minutes (building the AI index from Sanity content)
4. Watch for: 🚀 `HTR AI Brain v4.2.0 starting...` then `Index ready.`

Verify the Backend is Running

```
curl https://your-app.up.railway.app/health
```

Expected response:

```
{
  "status": "ok",
  "version": "4.2.0",
  "index_ready": true
}
```

If `index_ready` is `false`, the index is still building — wait a few more minutes and check again.

11. Vercel — Frontend

Vercel hosts the Next.js frontend. It serves the website, handles server-side rendering, and proxies API calls to the Railway backend.

Step 1 — Create an Account

- 1. Go to vercel.com → **Sign Up** → **Continue with GitHub** (recommended)

Step 2 — Import Your Repository

- 1. In Vercel dashboard → **Add New** → **Project**
- 2. Find your repository in the list → **Import**
- 3. On the configuration screen:
 - **Framework Preset:** Next.js (auto-detected)
 - **Root Directory:** Click **Edit** → type `frontend` → click **Continue**
 - **Build Command:** `npm run build` (default)
 - **Output Directory:** `.next` (default)
 - **Node.js Version:** 20.x (set under Build & Development Settings)
- 4. **Do not click Deploy yet** — add environment variables first

Step 3 — Add Environment Variables

In the same screen, scroll down to **Environment Variables**. Add every variable below. Click **Add** after each one.

Required — will break the site if missing

NEXT_PUBLIC_SUPABASE_URL	= https://[project-id].supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY	= eyJ... (anon key – safe for browser)
SUPABASE_SERVICE_ROLE_KEY	= eyJ... (service role – server only)
NEXT_PUBLIC_SANITY_PROJECT_ID	= fxz10xl7
NEXT_PUBLIC_SANITY_DATASET	= production
SANITY_API_TOKEN	= sk...
PYTHON_BACKEND_URL	= https://your-app.up.railway.app
INGEST_SECRET	= (same value you set in Railway)

Payment — required for subscriptions to work

```
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY = pk_test...
STRIPE_SECRET_KEY                   = sk_test...
STRIPE_PRICE_SUBSCRIBER_MONTHLY    = price...
```

```
STRIPE_PRICE_SUBSCRIBER_ANNUAL      = price_...
STRIPE_PRICE_PROFESSIONAL_MONTHLY   = price_...
STRIPE_PRICE_PROFESSIONAL_ANNUAL    = price_...
```

Webhooks — required for content sync and payment processing

```
SANITY_WEBHOOK_SECRET = (the random string you generated in Step 4 of Sanity
setup)
STRIPE_WEBHOOK_SECRET = whsec_... (you will get this in Step 13 – add it
then)
```

Monitoring — optional but recommended

```
NEXT_PUBLIC_SENTRY_DSN = https://...@sentry.io/... (frontend DSN)
SENTRY_AUTH_TOKEN      = ...
SENTRY_ORG             = your-org-slug
SENTRY_PROJECT         = htr-frontend
```

Step 4 — Deploy

Click **Deploy**. Vercel will:

- 1. Clone your repository
- 2. Install npm dependencies (`npm install`)
- 3. Run the build (`npm run build`)
- 4. Deploy to the global CDN

This takes 2–5 minutes. When complete, you will see a green checkmark and a URL like `htr-platform.vercel.app`.

Open that URL in your browser. The site should load.

Step 5 — Note Your Production URL

Copy the Vercel URL (e.g., `htr-platform.vercel.app`). You need to update two things now:

- 1. **Railway** → your service → **Variables** → update `FRONTEND_URL` to this URL → redeploy Railway
- 2. **Supabase** → Authentication → URL Configuration → update Site URL and Redirect URLs

12. Connect Everything — Environment Variables

Now that all services are deployed, do a final check that every variable is set correctly in both Vercel and Railway.

Vercel — Complete Variable Checklist

In Vercel → your project → **Settings** → **Environment Variables**:

```
REQUIRED (will show errors if missing):
❑ NEXT_PUBLIC_SUPABASE_URL
❑ NEXT_PUBLIC_SUPABASE_ANON_KEY
❑ SUPABASE_SERVICE_ROLE_KEY
❑ NEXT_PUBLIC_SANITY_PROJECT_ID
❑ NEXT_PUBLIC_SANITY_DATASET
❑ SANITY_API_TOKEN
❑ PYTHON_BACKEND_URL           ← must be your Railway URL, no trailing slash
❑ INGEST_SECRET                ← must match Railway exactly
```

- PAYMENTS:
- ☐

NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY
- ☐

STRIPE_SECRET_KEY
- ☐

STRIPE_WEBHOOK_SECRET

← add after completing Step 13

☐

STRIPE_PRICE_SUBSCRIBER_MONTHLY

☐

STRIPE_PRICE_SUBSCRIBER_ANNUAL

☐

STRIPE_PRICE_PROFESSIONAL_MONTHLY

☐

STRIPE_PRICE_PROFESSIONAL_ANNUAL

WEBHOOKS:

☐

SANITY_WEBHOOK_SECRET

MONITORING:

☐

NEXT_PUBLIC_SENTRY_DSN

☐

SENTRY_AUTH_TOKEN

☐

SENTRY_ORG

☐

SENTRY_PROJECT

Railway — Complete Variable Checklist

In Railway → your service → **Variables**:

- ☐

GROQ_API_KEY
- ☐

OPENAI_API_KEY
- ☐

ANTHROPIC_API_KEY
- ☐

SUPABASE_URL
- ☐

SUPABASE_SERVICE_ROLE_KEY
- ☐

SUPABASE_JWT_SECRET
- ☐

SUPABASE_DB_URL
- ☐

SANITY_PROJECT_ID
- ☐

SANITY_DATASET
- ☐

SANITY_API_TOKEN
- ☐

SANITY_API_VERSION

= 2023-10-01

☐

INGEST_SECRET

← must match Vercel exactly

☐

FRONTEND_URL

← your Vercel production URL

☐

SENTRY_DSN

← backend DSN (different from frontend)

☐

ENVIRONMENT

= production

After adding or changing any variable in Railway, Railway automatically redeploys. In Vercel, you must click **Redeploy** manually after adding variables (or the changes only apply to new deployments).

13. Post-Deployment — Webhooks & Final Setup

Now that both services have URLs, complete the webhook configuration.

Webhook 1 — Stripe → Your Platform

This allows Stripe to notify your platform when payments succeed or fail, so subscriptions are granted automatically.

1. In Stripe dashboard → **Developers** → **Webhooks** → **Add endpoint**
2. **Endpoint URL:** `https://your-vercel-domain.vercel.app/api/stripe/webhook`
3. **Events to send** — click **Select events** and add these four:

☐

`checkout.session.completed`

☐

`customer.subscription.updated`

☐

`customer.subscription.deleted`

☐

`invoice.payment_failed`
4. Click **Add endpoint**
5. On the webhook detail page → **Signing secret** → click **Reveal**
6. Copy the value (starts with `whsec_`)

7. Go to Vercel → Settings → Environment Variables → add:

```
STRIPE_WEBHOOK_SECRET = whsec_...
```

8. Redeploy Vercel (Deployments → the latest → Redeploy)

Webhook 2 — Sanity → Your Platform

This allows Sanity to notify your platform when content is published, triggering an AI re-index.

- 1. Go to sanity.io/manage → your project → **API** → **Webhooks** → **Add**
- 2. Configure:

```
Name:          HTR Content Sync
URL:           https://your-vercel-domain.vercel.app/api/webhooks/sanity
Dataset:       production
Trigger on:    Create, Update, Delete
HTTP method:  POST
Secret:        (the SANITY_WEBHOOK_SECRET value you set in Vercel)
Include drafts: No
```

3. Click **Save**

Grant Yourself Admin Access

Your own user account needs the **admin** role to access the admin dashboard.

- 1. Sign up on your platform at [/signup](#)
- 2. Verify your email
- 3. Go to Supabase → **SQL Editor** → run:

```
-- Replace the email with yours
INSERT INTO public.user_roles (user_id, role)
SELECT id, 'admin'
FROM auth.users
WHERE email = 'your@email.com'
ON CONFLICT (user_id, role) DO NOTHING;
```

- 4. Log out and back in on the platform
- 5. Navigate to [/admin](#) — you should see the admin dashboard

Trigger the First AI Index

The AI Analyst needs to index your Sanity content before it can answer questions.

- 1. Go to <https://your-domain.com/admin/ingest>
- 2. Click **Trigger Ingest**
- 3. Watch the status — it changes from **queued** → **running** → **completed**
- 4. This takes 5–15 minutes on first run

Or via command line:

```
curl -X POST https://your-backend.up.railway.app/api/ingest \
-H "Authorization: Bearer YOUR_INGEST_SECRET"
```

14. Verify Everything Works

Run through this checklist after deployment. Each item tests a different part of the system.

Basic Site

- ☐ Visit your production URL → homepage loads
- ☐ Navigation works (click pillars, research lab, etc.)
- ☐ Dark mode toggle works

Authentication

- ☐ `/signup` → create a test account → email verification arrives
- ☐ `/login` → log in with the test account
- ☐ `/account` → profile page loads

Database

- ☐ After login, your profile appears in Supabase → Table Editor → `profiles`
- ☐ Your role appears in `user_roles` table (should show `free`)

AI Analyst

- ☐ Open the right sidebar → type a question → response streams in
- ☐ If error: check Railway logs and health endpoint

```
curl https://your-backend.up.railway.app/health
# "index_ready" must be true
```

Payments (Test Mode)

- ☐ Click **Subscribe** → Stripe checkout page opens
- ☐ Use test card: `4242 4242 4242 4242`, any future expiry, any CVC
- ☐ After checkout → redirected back → account shows subscriber role
- ☐ In Supabase `user_roles` table → your test account has `subscriber` role

Content Sync

- ☐ Go to Sanity Studio at `/studio`
- ☐ Edit and publish any piece of content
- ☐ Check Railway logs → should see ingest job triggered within ~30 seconds

Admin Dashboard

- ☐ Navigate to `/admin` → admin dashboard loads
- ☐ `/admin/ingest` → trigger button works

15. Custom Domain Setup

To use your own domain (e.g., `healthtransformationreview.com`) instead of the Vercel URL:

Step 1 — Add Domain to Vercel

1. In Vercel → your project → **Settings** → **Domains**
2. Click **Add** → type your domain → **Add**
3. Vercel shows you DNS records to add

Step 2 — Configure Your DNS

Go to your domain registrar (GoDaddy, Namecheap, Cloudflare, etc.) → DNS settings:

If using a root domain (`healthtransformationreview.com`):

```
Type: A
Name: @
Value: 76.76.21.21    (Vercel's IP – Vercel will show you the exact value)
```

If using a subdomain (`app.healthtransformationreview.com`):

```
Type: CNAME
Name: app
Value: cname.vercel-dns.com
```

DNS changes take 5–60 minutes to propagate.

Step 3 — Update All Service Configurations

Once your domain is live, update these:

- 1. **Supabase** → Authentication → URL Configuration → Site URL → `https://yourdomain.com`
- 2. **Supabase** → Redirect URLs → add `https://yourdomain.com/**`
- 3. **Railway** → Variables → `FRONTEND_URL` → `https://yourdomain.com`
- 4. **Stripe** → Webhooks → update endpoint URL to `https://yourdomain.com/api/stripe/webhook`
- 5. **Sanity** → Webhooks → update URL to `https://yourdomain.com/api/webhooks/sanity`
- 6. **Vercel** → Environment Variables → update any hardcoded domain references

Step 4 — Go Live with Stripe

When you are ready for real payments:

- 1. Stripe dashboard → toggle from **Test mode** to **Live mode** (top right)
- 2. In live mode → **Developers** → **API keys** → copy the live keys (`pk_live_...`, `sk_live_...`)
- 3. In live mode → recreate your products and prices (Stripe test and live are separate)
- 4. In live mode → recreate the webhook endpoint with your production domain
- 5. Update Vercel environment variables with the live keys and live webhook secret
- 6. Redeploy Vercel

16. Troubleshooting

Site loads but AI chat returns an error

Check: Is Railway running?

```
curl https://your-backend.up.railway.app/health
```

If you get `Connection refused` or a timeout, the Railway backend is down. Check Railway logs.

If you get `{"status":"ok","index_ready":false}`, the AI index is still building. Wait and check again.

Login works but page redirects back to login

Check: Supabase Redirect URLs. The exact URL of your deployment must be in the allowed list.

Go to Supabase → Authentication → URL Configuration → make sure your Vercel URL is listed under **Redirect URLs** with a `/**` wildcard.

Stripe checkout completes but user still shows as Free

Check: Is the Stripe webhook configured and is `STRIPE_WEBHOOK_SECRET` correct in Vercel?

In Stripe dashboard → Developers → Webhooks → your endpoint → **Recent deliveries**. You should see a `checkout.session.completed` event. If it shows a red ✖, click it to see the error response from your server.

Most common cause: `STRIPE_WEBHOOK_SECRET` in Vercel doesn't match what Stripe shows. Reveal the signing secret in Stripe and paste it fresh into Vercel.

AI Analyst gives irrelevant answers

The knowledge base needs to be indexed. Go to `/admin/ingest` and trigger a re-index. First index takes 5–15 minutes.

Railway keeps restarting

Check Railway logs for the error. Common causes:

- **Missing environment variable** — look for `KeyError` or `None` type errors in logs
- **SUPABASE_DB_URL wrong** — pgvector connection fails at startup
- **OPENAI_API_KEY invalid** — embedding model fails during index build

Build fails on Vercel

Click the failed deployment → **View Build Logs** → scroll to the error.

Common causes:

- TypeScript error — fix the type error in the code
- Missing environment variable at build time — add the variable in Vercel settings, ensure it is set for the **Production** environment
- `npm install` failure — check `package.json` for missing packages

"pgvector extension not found" in Railway logs

Run this in Supabase SQL Editor:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Then restart Railway.

Quick Reference — All Credentials Location

Credential	Where to Find It
NEXT_PUBLIC_SUPABASE_URL	Supabase → Project Settings → API → Project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase → Project Settings → API → anon/public
SUPABASE_SERVICE_ROLE_KEY	Supabase → Project Settings → API → service_role
SUPABASE_JWT_SECRET	Supabase → Project Settings → API → JWT Settings
SUPABASE_DB_URL	Supabase → Project Settings → Database → Connection string (URI)
SANITY_API_TOKEN	sanity.io/manage → your project → API → Tokens
GROQ_API_KEY	console.groq.com → API Keys
OPENAI_API_KEY	platform.openai.com → API keys

Credential	Where to Find It
ANTHROPIC_API_KEY	console.anthropic.com → API Keys
STRIPE_SECRET_KEY	Stripe dashboard → Developers → API keys
STRIPE_WEBHOOK_SECRET	Stripe dashboard → Developers → Webhooks → your endpoint → Signing secret
STRIPE_PRICE_*	Stripe dashboard → Product catalog → your product → price ID
NEXT_PUBLIC_SENTRY_DSN	sentry.io → your frontend project → Settings → Client Keys (DSN)
SENTRY_AUTH_TOKEN	sentry.io → Settings → Auth Tokens
PYTHON_BACKEND_URL	Railway dashboard → your service → Settings → Domains
INGEST_SECRET	You generate this with <code>openssl rand -base64 32</code>
SANITY_WEBHOOK_SECRET	You generate this with <code>openssl rand -base64 32</code>